

PulseCRM

Week 2 Sprint Document

Authentication, RBAC & Organization Setup

Technocolabs Softwares Pvt. Ltd.

Full Stack Development Internship Program — 2 Interns

Sprint Summary

Field	Detail
Week	Week 2 of 8
Sprint Theme	Authentication, RBAC & Organization Setup
Prerequisite	Week 1 scaffolding (backend/frontend running, ER diagram & Prisma schema v1 approved)
Team	2 Full Stack Interns — both contribute across frontend & backend

Sprint Goal

Build a complete, secure authentication system with JWT access/refresh tokens, email verification, and password recovery — and establish the Company → Team → User organizational structure with Role-Based Access Control (Admin / Sales Manager / Sales Rep) enforced across the API.

Business Objectives

- Enable secure, multi-role login for the CRM platform
- Establish the organizational hierarchy (Company → Team → User) that all future modules depend on
- Lay the permission foundation so Contacts/Deals/Tasks (Weeks 3–5) can be correctly scoped by role

Learning Objectives

- Implement a production-style JWT access + refresh token flow
- Design and apply RBAC middleware in Express
- Practice secure password handling (bcrypt hashing, never storing plaintext)
- Build email-based verification and password-reset flows

Features Delivered This Week

- User registration & login
- JWT access token + refresh token rotation
- Email verification on signup
- Forgot password / reset password flow
- Role-Based Access Control (Admin / Sales Manager / Sales Rep)
- Company & Team creation
- Admin-driven user invitation flow

Tasks by Category

UI/UX Tasks

- Login screen (fully wired to backend)
- Register screen
- Forgot Password screen
- Reset Password screen
- Email verification pending / success screen
- Role-aware sidebar navigation (shows/hides items by role)
- Team/User management screen (Admin only)

Database Tasks

- Finalize and migrate Company, Team, and User Prisma models
- Add fields: emailVerified, verificationToken, resetToken, resetTokenExpiry, refreshTokenHash
- Seed script to create an initial Admin user for local development

Backend Tasks

- Auth service: register, login, refresh, logout
- Password hashing with bcrypt
- JWT signing/verification utility module
- Email service integration (nodemailer or provider) for verification & reset emails
- RBAC middleware (role guard) applied to protected routes
- Company/Team CRUD endpoints
- User invitation endpoint (Admin invites a new team member by email)

Frontend Tasks

- Build auth pages: Login, Register, Forgot Password, Reset Password
- Protected route wrapper component
- Redux Toolkit auth slice (user, tokens, role, isAuthenticated)
- Axios interceptor: attach access token to requests, auto-refresh on 401
- Role-based conditional rendering for navigation and pages

API Development

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/refresh
- POST /api/auth/logout
- GET /api/auth/verify-email/:token
- POST /api/auth/forgot-password
- POST /api/auth/reset-password/:token
- POST /api/companies

- POST /api/teams
- POST /api/users/invite

API Integration

- Wire frontend auth forms (React Hook Form + Zod) to backend endpoints via React Query mutations
- Wire protected routing and role-aware navigation to live auth state

Testing

- Jest unit tests: password hashing, JWT utilities, RBAC guard logic
- Supertest integration tests: register → verify → login → refresh → forgot → reset flows
- Postman collection covering the full auth flow end-to-end

Bug Fixing

- Resolve issues surfaced during Week 1 scaffold review
- Cross-review each other's auth implementation for edge cases

Security

- Passwords hashed with bcrypt — never stored or returned in plaintext
- JWT secrets read from environment variables, never hardcoded
- Refresh tokens stored hashed in the database and rotated on each use
- Rate limiting applied to login and forgot-password endpoints
- Zod input validation on all auth routes

Performance Optimization

- Index on User.email for fast lookup during login
- Avoid N+1 queries when fetching Company/Team/User relations

Git Tasks

- Target: 30–40 meaningful commits per intern this week (full feature week)
- 3–5 Pull Requests: auth-backend, auth-frontend, rbac-middleware, org-setup
- Feature branches for every task; mandatory code review before merge

Pair Programming

- Pair on RBAC middleware design — both interns co-design the permission matrix together

Code Review

- Cross-review every PR using a checklist: security, test coverage, error handling, consistent naming

Documentation

- Update Swagger/OpenAPI spec with all new auth & org endpoints
- Publish the GitBook Week 2 sprint page
- Update README.md with new auth-related environment variables

Daily Monday–Friday Plan

Day	Focus
Mon	Sprint planning; finalize Company/Team/User schema; run migration
Tue	Backend auth core — register, login, JWT issuance, bcrypt hashing
Wed	Refresh token rotation, email verification, forgot/reset password flows
Thu	RBAC middleware + Company/Team/User invite endpoints; begin frontend auth pages
Fri	Frontend integration, test suite, Swagger update, GitBook page, sprint review demo

Deliverables

- Fully working authentication system (register / login / refresh / logout)
- Email verification and forgot/reset password flows
- RBAC enforced server-side on all protected routes
- Company/Team/User management with working Admin invite flow
- Automated test suite covering all auth flows
- Updated Swagger documentation and published GitBook Week 2 page

Mentor Review Checklist

- ■ Passwords hashed, never returned in any API response
- ■ JWT access + refresh flow works correctly; refresh token rotates on use
- ■ Email verification correctly gates/flags unverified accounts
- ■ Forgot/reset password flow works end-to-end
- ■ RBAC middleware blocks unauthorized roles at the API level (not just hidden in UI)
- ■ Company/Team/User invite flow is functional
- ■ Jest + Supertest suite passing
- ■ Swagger docs updated and accurate
- ■ Both interns show equal contribution across frontend and backend

Client Demo

Live walkthrough: register a new user → verify email → log in → trigger forgot password → reset password → log in again. Then demonstrate RBAC by showing an Admin view vs. a Sales Rep view with different navigation and data access, followed by an Admin inviting a new team member.

Acceptance Criteria

- All auth endpoints are functional and covered by tests
- RBAC is enforced server-side, not only hidden in the UI
- No plaintext passwords or secrets appear in code, logs, or API responses
- Frontend successfully authenticates end-to-end against the backend

Learning Outcomes

- Understand and implement a secure, production-style JWT authentication system
- Learn practical RBAC design patterns for multi-role applications
- Gain experience building multi-tenant organizational data structures
- Hands-on experience integrating a transactional email service